

TRABALLO FIN DE GRAO
GRAO EN CIENCIA E ENXEÑARÍA DE DATOS

Visualizando el aprendizaje máquina: de la abstracción al entendimiento

Estudiante	Héctor Aldao Amoedo
Dirección:	Alejeandro Rodriguez Árias Samuel Magaz Romero

A Coruña, junio de 2026.

Dedicatoria¿?

Agradecimientos

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper. ¿?

Resumen

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Abstract

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Palabras clave:

- First itemtext
- Second itemtext
- Last itemtext
- First itemtext
- Second itemtext
- Last itemtext
- First itemtext

Keywords:

- First itemtext
- Second itemtext
- Last itemtext
- First itemtext
- Second itemtext
- Last itemtext
- First itemtext

Índice general

1	Introducción	1
1.1	Contexto y motivación	1
1.1.1	Definición del Problema	1
1.1.2	Objetivos	2
1.1.3	Alcance y Limitaciones	2
1.1.4	Metodología de Trabajo	2
1.1.5	Estructura de la Memoria	2
2	Estado del Arte	3
3	Herramientas y técnicas	7
3.1	Godot como motor de desarrollo	7
3.2	Control de versiones y despliegue	10
4	Metodología y planificación	12
5	Desarrollo	13
5.1	Prototipo: versión 0.1	13
5.1.1	Primeras escenas	14
5.1.2	Exportación web con GitHub Pages	14
5.2	Árbol de decisión: versión 0.2	15
5.2.1	Primera arquitectura del árbol	15
5.2.2	Reorganización de la escena y primeros elementos explicativos	16
5.2.3	Animación de la construcción del árbol	19
5.2.4	Gráficas y casos explicativos en la ventana	20
5.2.5	Primer panel de selección de datos	21
5.2.6	Carga de archivos y navegación por pasos	23
5.2.7	Primera evaluación visual del árbol	24

6 Conclusiones	27
7 Trabajo futuro	28
A Material adicional	30
Bibliografía	33

Índice de figuras

2.1	Sección interactiva de la página web de 3Blue1Brown.	5
2.2	Página web TensorFlow Playground.	5
3.1	Entorno de desarrollo de Godot.	8
3.2	Diagrama general de la arquitectura de Godot.	9
5.1	Versión preliminar del menú principal.	14
5.2	Primera escena de prueba del árbol. Arriba a la derecha hay dos botones que sirven para guardar y cargar un árbol ya creado. Abajo a la izquierda de ve una sencilla estructura conforma de árbol, con los nodos rojos con un texto de ejemplo conectados por líneas negras. Sobre uno de los nodos, se ve el menú que permite añadirle un hijo o eliminarlo.	17
5.3	Primer menú de control del entrenamiento algorítmico, con los botones para iniciar el entrenamiento y avanzar al siguiente paso antes de traducir la interfaz y aplicar los temas visuales definitivos.	17
5.4	Panel inicial de la escena del árbol para escoger entre creación manual y creación algorítmica.	17
5.5	Interfaz de entrenamiento del árbol tras traducir los controles. A la izquierda se muestra el botón para comenzar el entrenamiento y a la derecha el estado del árbol al avanzar con el botón de siguiente paso, con temas distintos para nodos internos y hojas.	18
5.6	Primera versión de la ventana informativa integrada en la vista del árbol, todavía con un mensaje inicial que indica al usuario que debe comenzar el entrenamiento.	18
5.7	Entrenamiento del árbol en el que se aprecian los nodos pendientes.	20
5.8	Ventana informativa en la que se dibuja la gráfica de la entropía de cada atributo.	22
5.9	Menú que aparece tras seleccionar el modo algorítmico para seleccionar el dataset de entrenamiento.	22

5.10	Vista del árbol en la que ahora está el botón de retroceder al paso anterior. . .	25
5.11	Fase de evaluación en la que se ven dos datos ya clasificados y uno en proceso de clasificación.	25

Índice de cuadros

Capítulo 1

Introducción

PRIMER capítulo da memoria, onde xeralmente se exporán as liñas mestras do traballo, os obxectivos, etc. Incluimos un par de exemplos de citas e de referencias

LA inteligencia artificial ...?

1.1 Contexto y motivación

Los algoritmos de aprendizaje máquina son técnicas matemáticas que escapan de las fronteras de lo que es entendible a simple vista por ¿una persona cualquiera.¿ Desde sus raíces más puramente matemáticas hasta su implementación tanto hardware como software.

Esto ya era cierto en los tiempos de los primeros algoritmos de aprendizaje ¿por refuerzo, árboles o qué?, pero con el surgir de los modelos de aprendizaje profundo, esta realidad es más palpable que nunca.

Bajo estas premisas es que se siente indispensable ¿proporcionar a la gente los materiales y métodos para que puedan comprender lo mejor posible esta tecnología cada vez más omnipresente¿?. ¿Y a de más el aprendizaje es mejor si es interactivo ¿?

1.1.1 Definición del Problema

Hoy en día las principales formas de aprender cómo funcionan los modelos de inteligencia artificial (sobre todo los clásicos, pero también los modernos) es a través de los siempre confiables libros y universidad, las más ancestrales fuentes de conocimiento. Entre los formatos más actuales también encontramos los vídeos divulgativos en plataformas online como Youtube, donde la posibilidad de que todo el mundo comparta su contenido ha dado lugar a ¿muchas oportunidades de ver las interpretaciones visuales de otras personas¿?.

Sin embargo ¿aún no existe ninguna herramienta pedagógica que permita interactuar en tiempo real con los algoritmos.¿ ¿Programándolos se pueden ver sus resultados, su evolución en una gráfica, pero no palpar los que está sucediendo en las entrañas de la bestia mientras

mastica el equivalente a su comida: los datos, tanto en entrenamiento como en inferencia;?
((.....))

1.1.2 Objetivos

((Crear la aplicación - paso 1, hacerla - paso 2, exportarla
Publicarla para que todo el mundo la pueda disfrutar))
((creo qe hay q se rmás concreto))

Los objetivos que se plantean en este trabajo comienzan por plantearse qué algoritmos de IA son los más interesantes para ¿explicar¿. Esta es, en última instancia, una cuestión subjetiva, por lo que en este trabajo se abordó esta decisión mirando ¿de cara¿ a dos de los mayores paradigmas de la IA: la simbólica y el deeplearning. ((no se si decir de forma pragmática y decir que tenía en mente las redes neuronales, y que los árboles se parecían lo suficiente, o poner un poco de fantasía de "los árboles, uno los primeros amigos del hombre en cuánto a lo que hacer que una máquina aprenda a jugar al ajedrez..." (lo

De ambos se decidió que los mejores candidatos para protagonizar una explicación que busque asentar los fundamentos eran las versiones más fundamentales de cada uno. Del lado de la IA simbólica está el árbol de decisión ((planteado por bla bla bla en blabal)), conocido por ser ... white box Y en el deeplearning, el ladrillo fundacional, la red neuronal ((lo mismo de histoia...))

1.1.3 Alcance y Limitaciones

1.1.4 Metodología de Trabajo

1.1.5 Estructura de la Memoria

Estado del Arte

LA inteligencia artificial y el aprendizaje automático se apoyan en algoritmos cuyo funcionamiento interno no siempre resulta fácil de visualizar. Conceptos como el cálculo de una frontera de decisión o la actualización iterativa de los parámetros de un modelo pueden resultar abstractos cuando se estudian únicamente desde una formulación teórica. Esta dificultad se acentúa en el aprendizaje profundo, donde intervienen elementos como funciones de activación, pesos, sesgos, propagación hacia delante, retropropagación o gradientes. Por ello, se han desarrollado recursos educativos orientados a hacer más comprensible el comportamiento de estos modelos mediante explicaciones visuales y simulaciones interactivas. En el contexto de este trabajo, resultan especialmente relevantes las herramientas que permiten comprender el proceso de aprendizaje de los modelos, la relación entre sus componentes internos y la forma en la que una decisión algorítmica se transforma en un resultado observable.

Uno de los recursos divulgativos más conocidos para introducir las redes neuronales es la serie de vídeos *Neural networks* [1]. Esta fue publicada en el canal de YouTube *3Blue1Brown*. Este proyecto cuenta también con una página web [2], cuya principal aportación pedagógica es la posibilidad de interactuar con una red neuronal entrenada con el conjunto de datos MNIST [3]. La herramienta permite observar en tiempo real cómo la red reacciona a los cambios en el dato de entrada, representado mediante una superficie de dibujo en la que cada píxel se corresponde con una neurona de la capa de entrada, como se aprecia en la Figura 2.1.

Este tipo de recurso resulta valioso para desarrollar intuición, ya que reduce la barrera de entrada matemática y proporciona una primera aproximación visual al funcionamiento de una red. También es adecuado para motivar y contextualizar el problema gracias a una explicación apoyada en recursos visuales claros. Su formato, en cambio, conserva un carácter fundamentalmente expositivo: en la página web, dos de las diez lecciones incorporan un componente interactivo, y dicha interacción se orienta siempre a la relación entre la entrada y la salida de una red ya entrenada. El estudiante trabaja con una explicación limitada al conjunto MNIST y sin capacidad de experimentar de forma directa con la arquitectura del modelo. Esta

restricción deja abierta la necesidad de recursos más activos, en los que sea posible analizar el efecto de cada decisión de diseño sobre el comportamiento de una red neuronal.

Frente al formato audiovisual, existen herramientas que permiten manipular redes neuronales en tiempo real. Un ejemplo es *A Neural Network Playground* [4], una aplicación que permite configurar una red neuronal sencilla desde el navegador, seleccionar conjuntos de datos sintéticos, modificar hiperparámetros y observar cómo evoluciona la frontera de decisión durante el entrenamiento. En la Figura 2.2 se puede visualizar la página web.

A diferencia de los recursos puramente expositivos, la relevancia de TensorFlow Playground reside en que conecta elementos habitualmente tratados de forma abstracta con consecuencias observables. Modificar directamente la arquitectura del modelo (número de capas y neuronas) o sus hiperparámetros (tasa de aprendizaje y regularización) altera de manera inmediata el proceso de ajuste. Este enfoque interactivo facilita que el estudiante relacione conceptos críticos como el sobreajuste, la generalización o la convergencia con resultados palpables. La herramienta se centra principalmente en la frontera de decisión y en la pérdida, sin profundizar de forma detallada en operaciones internas como el cálculo de gradientes o la propagación de errores capa a capa.

Esta misma orientación hacia la experimentación aparece en herramientas centradas en arquitecturas más específicas. *GAN Lab* [5] permite experimentar en el navegador con redes generativas adversarias. Este tipo de modelo introduce una dinámica de aprendizaje distinta a la de una red supervisada clásica, ya que enfrenta dos redes: un generador, que intenta producir muestras plausibles, y un discriminador, que intenta distinguir entre datos reales y generados. La aportación principal de GAN Lab es representar visualmente una interacción compleja como la de un par de redes adversarias. La herramienta permite observar cómo cambian las distribuciones generadas durante el entrenamiento y cómo la competencia entre generador y discriminador afecta al resultado. Desde el punto de vista educativo, esto resulta útil para explicar que existen modelos de aprendizaje profundo con objetivos y esquemas de aprendizaje diferentes.

La limitación principal de *GAN Lab*, junto con la falta de detalle sobre ciertas operaciones internas, es que presupone una base de conocimiento mayor que la requerida para una introducción a las redes neuronales. Para comprender correctamente una GAN es necesario haber asimilado previamente ideas como función de pérdida, optimización, representación interna o entrenamiento iterativo. En consecuencia, *GAN Lab* es una herramienta valiosa para una fase posterior del aprendizaje, aunque no sustituye a una aplicación centrada en los fundamentos de una red neuronal básica.

En el ámbito de los algoritmos clásicos de aprendizaje automático, también existen propuestas para ilustrar el funcionamiento de los árboles de decisión. La herramienta web interactiva sobre árboles de decisión de Interactive Machine Learning [6] permite al usuario

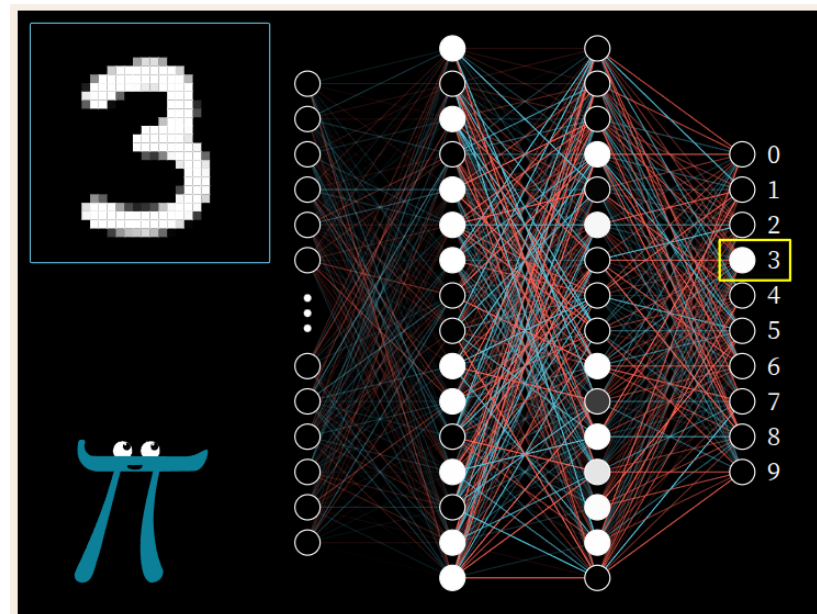


Figura 2.1: Sección interactiva de la página web de 3Blue1Brown.

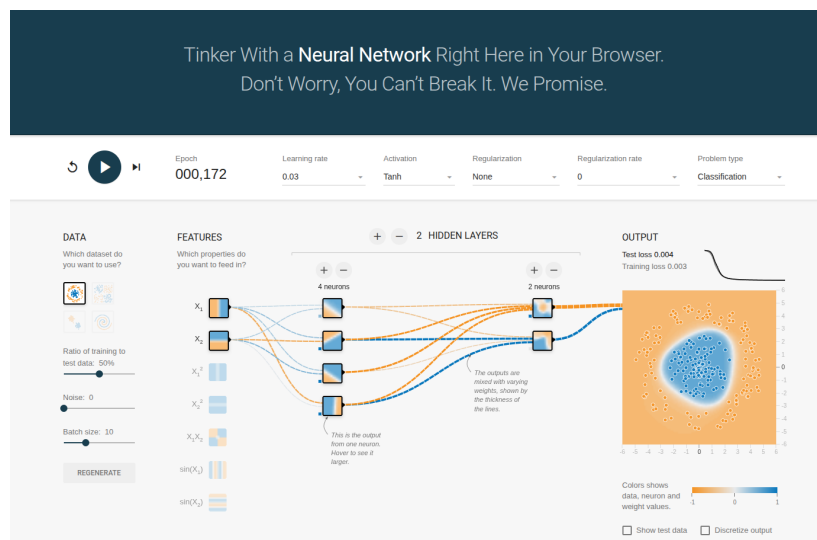


Figura 2.2: Página web TensorFlow Playground.

construir un árbol y observar el proceso de división de un conjunto de datos en tiempo real. Esta propuesta resulta interesante porque acerca al estudiante a la estructura jerárquica del modelo y al proceso de partición sucesiva de los datos. Su experiencia de uso, no obstante, es limitada y las explicaciones que acompañan a la simulación tienen poco desarrollo.

Aunque muestra los valores de distintas métricas, no acompaña dichos valores con una interpretación suficiente de su significado ni de su papel dentro del algoritmo. Esto reduce su utilidad como recurso formativo autónomo, especialmente para estudiantes que se aproximan por primera vez a los árboles de decisión.

En comparación con las redes neuronales, la disponibilidad de herramientas interactivas centradas en árboles de decisión es más reducida. La revisión realizada muestra una presencia mucho mayor de simuladores y visualizaciones dedicados a redes neuronales, tanto en recursos divulgativos como en herramientas web orientadas a la experimentación. Esta diferencia es especialmente visible cuando se buscan recursos que combinen visualización del modelo, explicación del criterio de partición y evaluación paso a paso.

Desde la perspectiva de este proyecto, la principal oportunidad se encuentra en combinar las fortalezas de estos recursos. Una herramienta educativa eficaz debería mantener la claridad visual de los recursos divulgativos y su atención a la base matemática, incorporar la manipulación directa de los simuladores y ofrecer suficiente rigor técnico para que lo presentado al usuario se corresponda con el funcionamiento real del algoritmo. También debería evitar que la interacción se limite a modificar hiperparámetros globales. Para entender una red neuronal resulta especialmente útil poder observar el flujo de datos, la contribución de los pesos y el efecto del entrenamiento sobre cada componente del modelo. En el caso de los árboles de decisión, la visualización debería mostrar cómo se selecciona cada atributo, cómo se divide el conjunto de datos y cómo se alcanza una clasificación final a partir de las características de entrada. En ambos modelos, esta visualización debe ir acompañada de explicaciones textuales que interpreten las métricas, los cálculos y las decisiones del algoritmo, de forma que la simulación no dependa únicamente de la observación visual.

Herramientas y técnicas

EL desarrollo de este trabajo se apoya en un conjunto de herramientas orientadas a construir una aplicación educativa, interactiva y ejecutable desde el navegador. La elección tecnológica del entorno de desarrollo tuvo en cuenta tanto criterios de implementación como la naturaleza del problema abordado: explicar modelos de aprendizaje automático mediante visualizaciones dinámicas, animaciones y ejemplos manipulables por el usuario. En consecuencia, se priorizaron tecnologías que facilitasen la construcción de interfaces interactivas, la validación de los algoritmos implementados y la publicación del resultado final en un entorno accesible. Godot permite exportar un mismo proyecto a distintas plataformas, como escritorio, móvil o web. En este caso, la versión web se escogió por su accesibilidad: el usuario puede abrir la aplicación desde un navegador, sin instalar software ni preparar un entorno de ejecución específico.

3.1 Godot como motor de desarrollo

La herramienta principal empleada fue Godot, un motor de desarrollo libre y multiplataforma orientado a la creación de aplicaciones interactivas [7]. Sus características resultan adecuadas para una aplicación educativa como la desarrollada en este trabajo. El proyecto requiere presentar información visual, responder a acciones del usuario, animar procesos paso a paso y mantener una estructura clara entre los distintos elementos de la interfaz. Estas necesidades encajan de forma natural con el modelo de escenas, nodos y señales que propone Godot [8].

No se si este párrafo anterior enlaza bien con este siguiente

Y si podeis porfavor revisar que se entienda bien la explicación de los nodos.

A lo largo de esta memoria se hablará en repetidas ocasiones de dos de las principales estructuras con las que se trabaja en Godot: las escenas y los nodos.

Los nodos son el elemento básico y más importante con el que se trabaja en el motor. Todo

elemento y subelemento que vaya a estar presente en tiempo de ejecución en el programa es un nodo. Y cada nodo cuenta, principalmente, con una serie de atributos, métodos y señales. Igual que en la programación orientada a objetos, los atributos son variables o constantes y los métodos funciones. Las señales son disparadores de funciones. Cuando son emitidas llaman a la función a la que están conectadas. Son una forma de conectar eventos entre diferentes nodos. Son el núcleo del paradigma de programación que promueve el motor: la programación dirigida por eventos.

Los atributos, métodos y señales de cada nodo vienen definidos por su clase. Godot dispone de un gran árbol de clases predefinidas que heredan atributos y métodos de sus clases padre. Las tres clases básicas son: el *Node* una clase vacía, la raíz del resto de nodos, que solo cuenta con los métodos mínimos para que funcione en el motor; el *Node2D* característico por contar con atributos referentes a la posición en un espacio bidimensional; el *Node3D* similar al anterior pero en un espacio tridimensional; y el *Control* pensado para los elementos de interfaz de usuario. Por encima de estas, el desarrollador siempre puede definir sus propias clases ampliando las ya existentes.

Los nodos por sí solos son clases abstractas que no existen en el programa ejecutable final por sí mismos. Para instanciarlos están las escenas. Estas son la materialización concreta del proyecto: los archivos que son guardados en disco y con los que se interactúa en el editor como se puede ver en la Figura 3.1. Están formadas, como mínimo, por un nodo raíz. De este nodo raíz pueden colgar nodos hijo, que a su vez pueden contar con sus propios nodos hijo, y así sucesivamente.

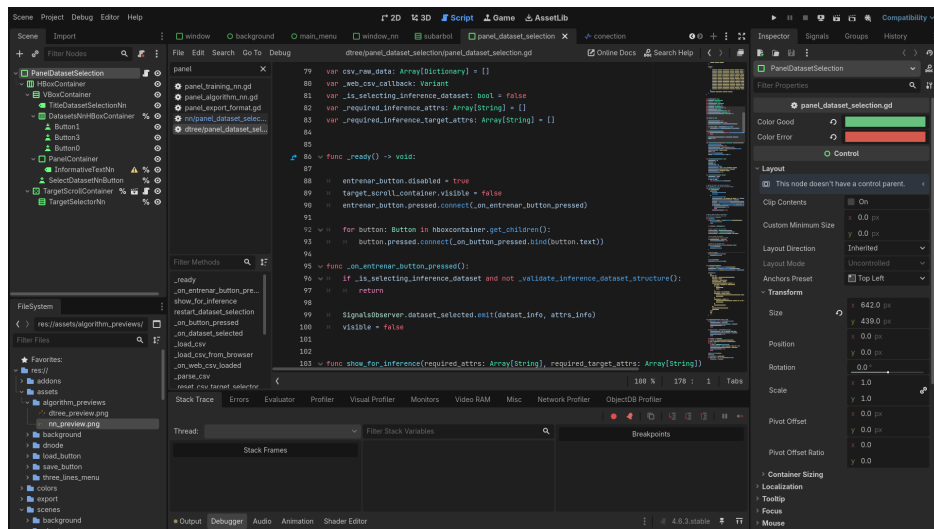


Figura 3.1: Entorno de desarrollo de Godot.

Hace falta que en el pie de foto de 3.1 describa en mayor profundidad lo que hay en pantalla?

Finalmente, las escenas, como en última instancia son árboles de nodos, pueden ser ins-

tanciadas dentro de otras escenas. Esto habilita una modularidad total, puesto que toda estructura de nodos que sea susceptible de ser reutilizada, puede guardarse como una escena para no tener que ser recreada.

Además, los nodos pueden tener asociado un script. Estos son código desde el que se puede interactuar, principalmente, con los atributos y métodos del nodo al que están asociados, así como con sus hijos. Pero no están limitados a estos elementos. Siempre se puede acceder al nodo raíz y, desde este, modificar cualquier propiedad global o acceder a cualquier nodo en la escena. Además, mediante la creación de *Autoloads* en el proyecto también se puede acceder a clases personalizadas en cualquier punto del código.

La Figura 3.2 muestra de forma esquemática esta organización y sitúa las escenas, los nodos, los scripts y los módulos dentro de la arquitectura general del motor.

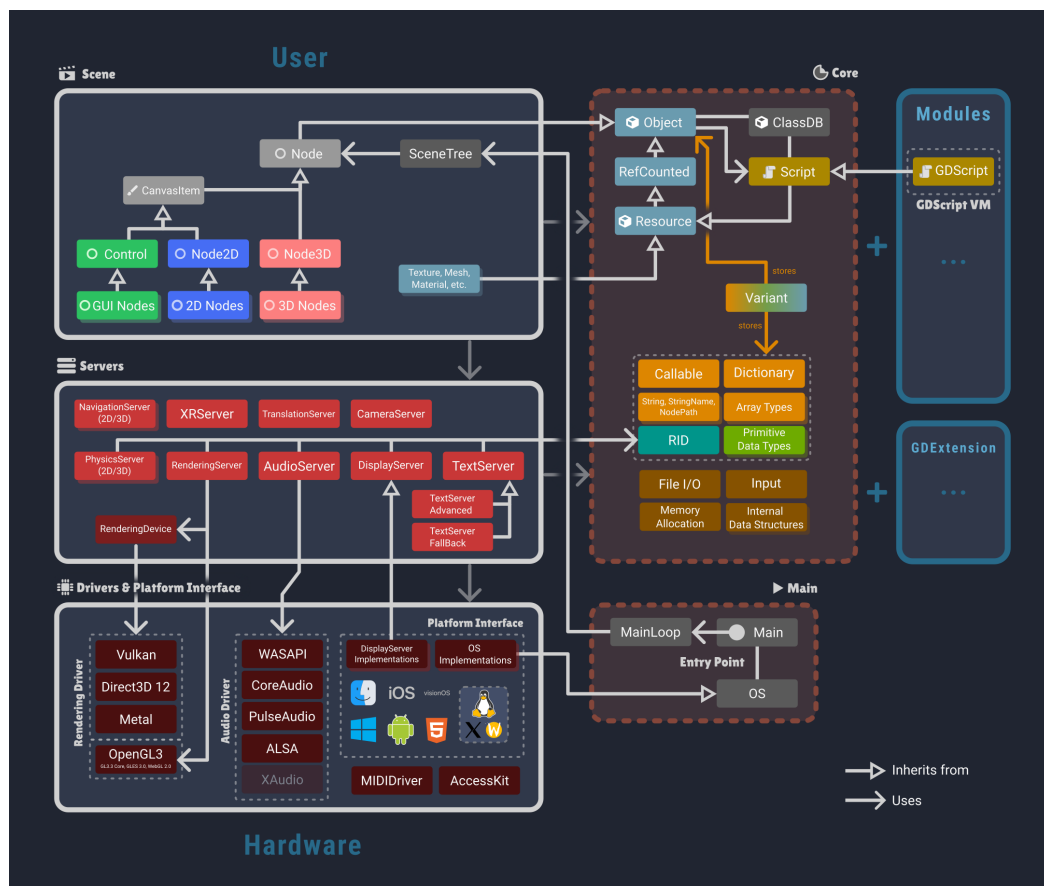


Figura 3.2: Diagrama general de la arquitectura de Godot.

La organización en escenas permite separar componentes de la aplicación con responsabilidades distintas, como las vistas de entrenamiento, los paneles explicativos, las representaciones gráficas y los controles de interacción. Este enfoque favorece una estructura modular en la que cada parte de la interfaz puede desarrollarse, probarse y reutilizarse con menor aco-

plamiento. El sistema de señales facilita la comunicación entre elementos de la aplicación sin imponer dependencias directas innecesarias, lo que resulta útil en un entorno en el que las acciones del usuario deben provocar cambios visuales y actualizaciones del estado interno de los modelos.

La manera en que Godot organiza las interfaces mediante nodos permite estructurar los elementos de los algoritmos tratados de forma sencilla. Un árbol de decisión puede mostrarse como una estructura jerárquica de nodos conectados, mientras que una red neuronal puede representarse mediante capas, neuronas y conexiones. Esta correspondencia conceptual ayuda a trasladar las estructuras internas del algoritmo a elementos visuales manipulables dentro de la aplicación.

Otro motivo importante para escoger Godot fue su sistema de exportación multiplataforma. El motor permite generar versiones para diferentes entornos a partir del mismo proyecto, incluida una versión HTML5 de la aplicación [9]. La exportación web se adoptó por la forma en la que se quería distribuir la herramienta: una aplicación educativa debe ser sencilla de abrir, compartir y probar. Publicarla como aplicación web evita exigir al usuario una instalación local y facilita el acceso desde un navegador. Esta característica es especialmente relevante en un proyecto con finalidad educativa, ya que reduce la fricción de uso y permite distribuir la herramienta como una aplicación web estática.

No tengo claro que ya se estén dando suficientes argumentos a favor de usar godot

3.2 Control de versiones y despliegue

Para la gestión del código fuente se utilizó Git, un sistema de control de versiones distribuido ampliamente utilizado [10]. Su uso permitió mantener un registro de los cambios y trabajar de forma incremental. Esto resultó útil durante el desarrollo, ya que la aplicación fue creciendo mediante pruebas sucesivas: primero se validó la interacción básica, después se incorporaron los algoritmos y más adelante se añadieron explicaciones y mejoras de interfaz. Disponer de un historial de cambios permitió avanzar con mayor seguridad y conservar puntos estables del proyecto mientras se realizaban modificaciones.

El repositorio se alojó en GitHub, lo que proporcionó almacenamiento remoto y una copia accesible del proyecto. Además, se utilizó GitHub Pages para publicar la versión web de la aplicación [11]. Este servicio permite desplegar sitios estáticos directamente desde un repositorio, por lo que encaja con la exportación web generada por Godot. La aplicación exportada se publica como un conjunto de archivos preparados para ejecutarse en el navegador, distinto del proyecto editable de Godot.

Esta fase tiene importancia porque marca la diferencia entre una aplicación que funciona dentro del entorno de desarrollo y una herramienta que puede utilizar cualquier persona desde

un enlace. En el editor, el proyecto se ejecuta en un contexto controlado, con acceso directo a las escenas, scripts y recursos locales. En el navegador, la aplicación depende de los archivos exportados, de las restricciones propias de la plataforma web y de la forma en la que el servicio de publicación sirve esos archivos. Por este motivo, el despliegue se trató como una parte más del desarrollo y no únicamente como el último paso administrativo.

La combinación de Godot y GitHub Pages permitió que el resultado final pudiese consultarse desde el navegador sin infraestructura de servidor adicional. Esto encaja con el objetivo de accesibilidad planteado al escoger la exportación web: el usuario solo necesita abrir una dirección, mientras que el proyecto mantiene un proceso de publicación suficientemente simple para ser actualizado durante el desarrollo.

Metodología y planificación

Capítulo 5

Desarrollo

En este capítulo se describe el proceso seguido para construir la aplicación. La explicación se plantea como una evolución por versiones. Comienza por comprobar que la idea es viable. Después se desarrolla el árbol de decisión, desde sus inicios hasta el resultado funcional con todas las características que se buscaban. Continúa con el desarrollo de red neuronal, pasando por todas sus fases de creación y mejora. Y termina con el pulido final de la interfaz y retoques a todas las características con las que cuenta el resultado final.

aquí "característica" lo estoy traduciendo de "feature", creo q está bien pero me está sonando algo raro

5.1 Prototipo: versión 0.1

El primer objetivo consiste en comprobar que la herramienta elegida sirve para crear una aplicación interactiva y publicarla como página web. Antes de invertir más tiempo en el desarrollo, es necesario verificar tres aspectos básicos: que Godot permite crear una interfaz sencilla, que se puede representar una estructura con forma de árbol y que el resultado puede ejecutarse desde el navegador.

Por ese motivo, la primera versión del proyecto es deliberadamente simple. Consiste en un menú inicial y en una escena donde el usuario puede crear pequeños elementos conectados entre sí. Al pulsar sobre uno de ellos aparece una opción para añadir un nuevo elemento debajo, de manera que poco a poco se forma una estructura parecida a un árbol. aquí asumo que cuando hablo de árbol se entiende que es la estructura de datos (por lo tanto que se ve "del revés" a como se ve un árbol real) y no un árbol real, no?? Esta prueba está todavía lejos de un árbol de decisión real. Su valor está en comprobar si la interacción principal de la aplicación tiene sentido.

5.1.1 Primeras escenas

La primera escena creada es el menú principal. En ese momento tiene un único botón, ya que su papel es servir como punto de entrada hacia la prueba del árbol. Incluir este menú desde el principio ayuda a pensar la aplicación como un conjunto de módulos y facilita su ampliación posterior. En la Figura 5.1 se ve esta primera versión.

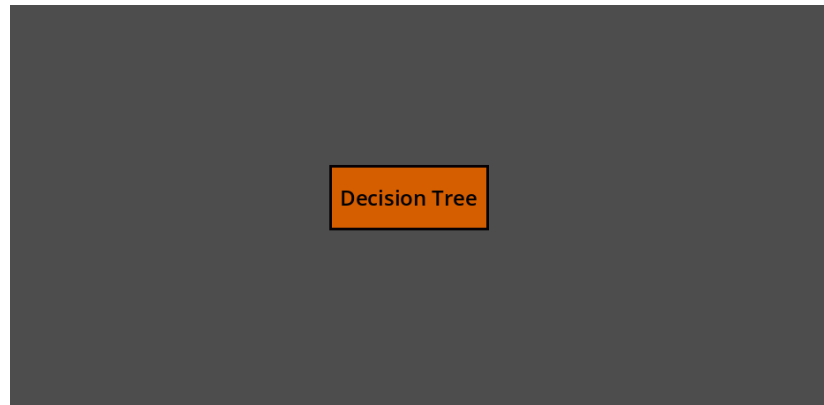


Figura 5.1: Versión preliminar del menú principal.

Al entrar en la escena del árbol aparece un único elemento inicial. Desde él se pueden crear nuevos elementos hijos, conectados por líneas y colocados bajo el anterior. También se añade la posibilidad de borrar partes del árbol, con el fin de comprobar que la estructura puede cambiar durante la ejecución sin que la escena deje de ser comprensible.

Esta primera prueba tiene una finalidad muy concreta: validar la interacción. Lo importante es comprobar que el usuario puede construir una jerarquía visual, que las conexiones se actualizan y que la pantalla sigue siendo manejable al crecer el número de elementos. En la Figura 5.2 se muestra cómo era esta escena inicial.

También se dejan visibles dos botones relacionados con cargar y guardar. En esta versión todavía no tienen una utilidad real. Su presencia sirve para reservar un espacio de la interfaz a una necesidad que ya se prevé: poder conservar un árbol y recuperarlo más adelante.

5.1.2 Exportación web con GitHub Pages

Una vez que la prueba interactiva funciona dentro de Godot, el siguiente paso lógico es llevarla al navegador. Esta comprobación es esencial, ya que la aplicación está pensada para ser consultada como una página web y también debe poder probarse fuera del editor de desarrollo.

La exportación se realiza correctamente con este proyecto inicial. El resultado se publica mediante GitHub Pages, usando la configuración del propio repositorio para indicar qué carpeta debe servir como página web. Con esto se confirma que el flujo completo es viable:

desarrollar en Godot, exportar el proyecto y publicarlo en una dirección accesible desde el navegador.

Esta prueba temprana reduce el riesgo del proyecto. Si la exportación web da problemas graves, hace falta replantear la tecnología antes de avanzar con los algoritmos. Al funcionar desde el inicio, el desarrollo puede continuar sobre una base más segura.

Añado aquí también algo de chicha sobre la exportación al ghpages?

5.2 Árbol de decisión: versión 0.2

El proyecto deja de ser un prototipo aislado cuando comienza el desarrollo del primer algoritmo: el árbol de decisión. A partir de este punto, el reto pasa a consistir en conseguir que el árbol se construya siguiendo las decisiones del algoritmo y que el usuario pueda entender cada paso.

5.2.1 Primera arquitectura del árbol

El primer cambio es separar la prueba visual inicial de una estructura preparada para crecer. El árbol tiene que convertirse en una vista capaz de representar tanto acciones del usuario como decisiones tomadas por el algoritmo.

Para ello se mantiene el menú principal como entrada de la aplicación y se crea una vista específica para el árbol. Esta vista debe resolver un problema práctico desde el principio: un árbol puede crecer hacia abajo y hacia los lados con mucha rapidez. Si todo se coloca en una pantalla fija, una parte del contenido queda fuera de la vista en cuanto aparecen varios niveles. Por esta razón se introduce una zona desplazable, de forma que el usuario pueda moverse por el árbol cuando su tamaño aumente.

También se revisa la forma de colocar los elementos. La versión inicial puede añadir hijos bajo un nodo y el resultado necesita más orden. Se adopta una distribución sencilla: los hijos se colocan por debajo de su padre y el padre queda centrado respecto a ellos. Esta decisión busca mantener una lectura clara y estable cada vez que el árbol cambia.

Otro aspecto importante es separar visualmente los elementos del árbol y las líneas que los unen. Esta separación hace que cada parte tenga una responsabilidad clara: unos elementos representan preguntas o resultados, y las líneas representan el camino seguido entre ellos. Gracias a esto, más adelante es más sencillo animar las ramas y mostrar sobre ellas el valor que conduce de un punto del árbol al siguiente.

En esta etapa se conserva también el modo manual. El objetivo principal es construir árboles de decisión, y el modo manual sigue siendo útil para probar la interfaz, comprobar el desplazamiento por la pantalla y detectar problemas de colocación sin depender todavía del algoritmo.

El siguiente avance es preparar el modo automático. En este modo, el usuario deja de añadir ramas a mano y pasa a controlar el avance del entrenamiento. La primera versión de esta interfaz es todavía muy simple, con botones en inglés, como se muestra en la Figura 5.3. Esta pantalla introduce una idea que se mantiene durante todo el desarrollo: el algoritmo avanza al ritmo marcado por el usuario.

Para conseguirlo, el algoritmo se organiza de forma incremental. La construcción completa se reparte en pasos pequeños: se guarda el punto en el que se encuentra el proceso y se resuelve una parte cada vez que el usuario pulsa el botón de avanzar. Esta decisión es clave para el propósito educativo de la aplicación, porque permite observar cómo se forma el árbol paso a paso.

La primera prueba de entrenamiento utiliza un conjunto de datos pequeño y controlado. Su objetivo es comprobar que el árbol se crea correctamente, que las ramas aparecen donde deben y que la interfaz se actualiza después de cada paso, como se aprecia en la Figura 5.3. En esta fase, lo más difícil es coordinar tres niveles al mismo tiempo: el estado interno del algoritmo, la estructura del árbol y lo que el usuario ve en pantalla.

5.2.2 Reorganización de la escena y primeros elementos explicativos

Cuando el modo automático empieza a funcionar, se reorganiza la escena para simplificarla. Algunas piezas que resultan útiles durante las pruebas iniciales dejan de ser necesarias, así que se reduce la cantidad de elementos intermedios. Esta limpieza facilita seguir avanzando sin arrastrar soluciones provisionales.

Uno de los problemas que aparece al reorganizar la vista es el centrado del árbol. Al crecer, el árbol puede quedar desplazado hacia una esquina, lo que dificulta seguir la construcción. Se ajusta entonces el comportamiento de la zona desplazable para que el contenido aparezca centrado y el usuario pueda moverse por él de forma natural. Esta mejora mantiene intacto el algoritmo y afecta mucho a la experiencia de uso.

La escena del árbol empieza a funcionar como un pequeño flujo guiado. Primero se muestra un panel para escoger entre crear el árbol manualmente o dejar que lo construya el algoritmo. Después, según la opción elegida, aparecen los controles correspondientes. En la Figura 5.4 se muestra este panel inicial.

En esta misma etapa se empieza a cuidar más la apariencia de los nodos. Los puntos que representan una decisión y los puntos que representan una clasificación final pasan a tener estilos diferentes. Esta distinción visual ayuda a leer el árbol sin depender tanto del texto: algunas partes indican que todavía hay una pregunta que responder, y otras indican que se llega a una conclusión.

También se traduce la interfaz al castellano. Los botones iniciales pasan a llamarse “Empezar entrenamiento” y “Siguiente paso”. Este cambio es pequeño desde el punto de vista

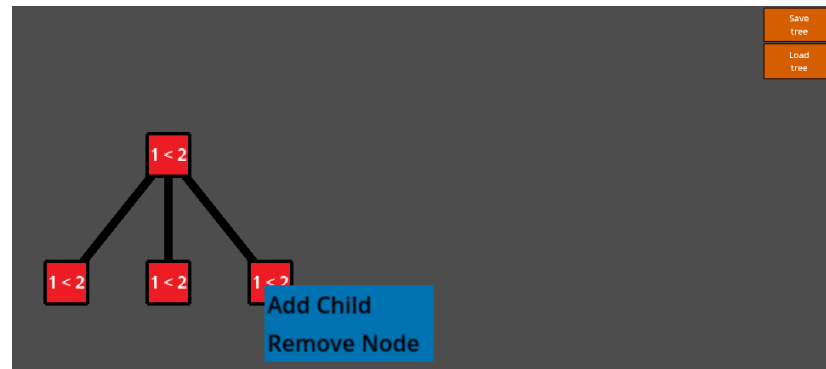


Figura 5.2: Primera escena de prueba del árbol. Arriba a la derecha hay dos botones que sirven para guardar y cargar un árbol ya creado. Abajo a la izquierda se ve una sencilla estructura conforma de árbol, con los nodos rojos con un texto de ejemplo conectados por líneas negras. Sobre uno de los nodos, se ve el menú que permite añadirle un hijo o eliminarlo.

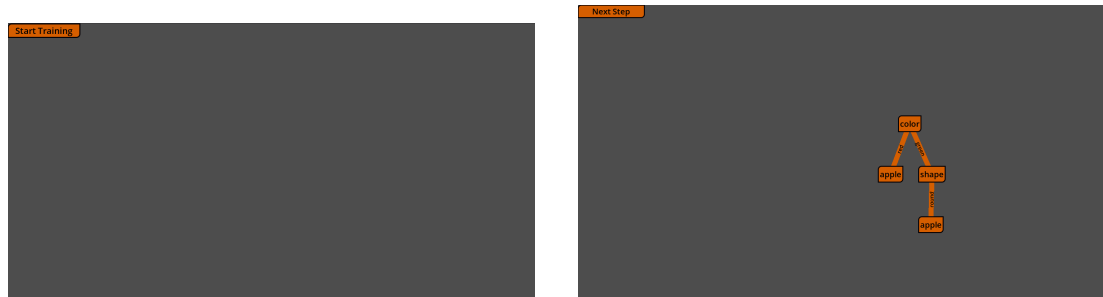


Figura 5.3: Primer menú de control del entrenamiento algorítmico, con los botones para iniciar el entrenamiento y avanzar al siguiente paso antes de traducir la interfaz y aplicar los temas visuales definitivos.



Figura 5.4: Panel inicial de la escena del árbol para escoger entre creación manual y creación algorítmica.

técnico y resulta importante para que la aplicación sea coherente con el resto del proyecto. En la Figura 5.5 se ve la interfaz después de esta adaptación y de los primeros cambios visuales.

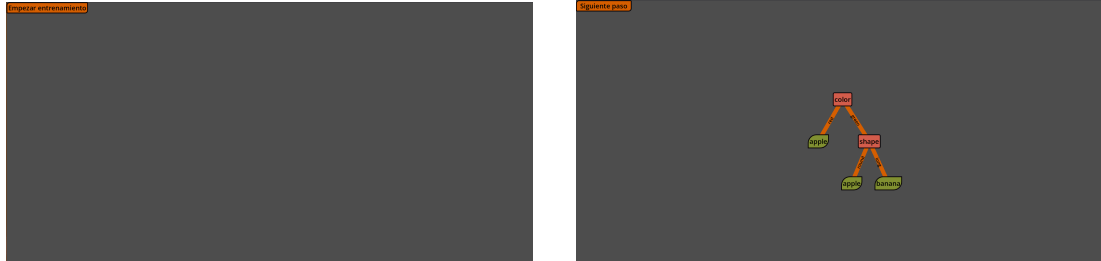


Figura 5.5: Interfaz de entrenamiento del árbol tras traducir los controles. A la izquierda se muestra el botón para comenzar el entrenamiento y a la derecha el estado del árbol al avanzar con el botón de siguiente paso, con temas distintos para nodos internos y hojas.

El siguiente paso natural es añadir una ventana informativa. Hasta ese momento, el árbol muestra qué se está creando y apenas explica por qué se toma cada decisión. La ventana nace para cubrir esa carencia: debe acompañar a la construcción del árbol con una explicación escrita del paso actual.

Su primera versión contiene un mensaje inicial que invita al usuario a comenzar el entrenamiento. Más adelante se convierte en un componente propio, separado de la vista principal, porque va a necesitar textos distintos según el caso y debe poder actualizarse cada vez que avance el algoritmo. En la Figura 5.6 se puede ver esta primera integración.

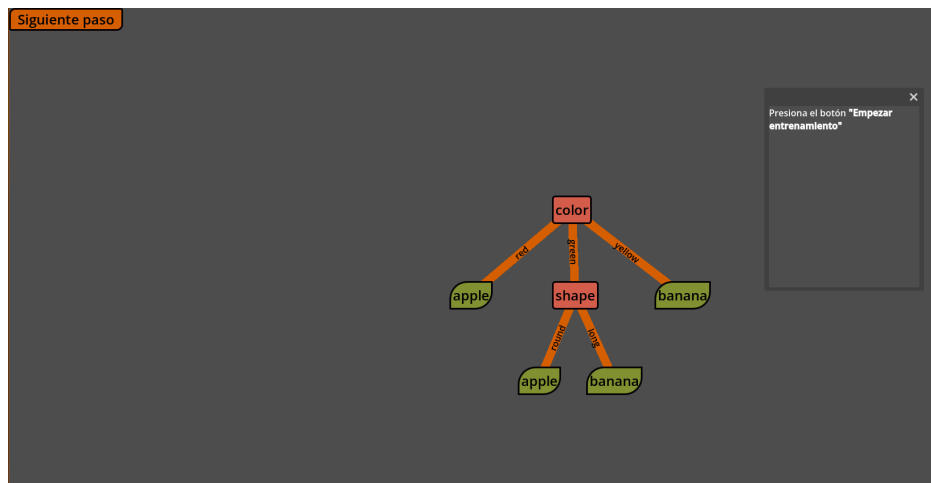


Figura 5.6: Primera versión de la ventana informativa integrada en la vista del árbol, todavía con un mensaje inicial que indica al usuario que debe comenzar el entrenamiento.

Los primeros textos de la ventana explican tres situaciones frecuentes. La primera es la creación de un nodo de decisión, cuando el algoritmo todavía tiene que escoger por qué característica dividir los datos. La segunda es la creación de una hoja porque todos los ejemplos

que llegan a ese punto tienen la misma respuesta. La tercera es la creación de una hoja por mayoría, usada cuando ya no hay más preguntas útiles y se debe escoger la respuesta más frecuente.

Durante estas pruebas se detecta que el texto necesita una preparación previa. Algunas listas aparecen con un formato propio de programación y poco adecuado para la lectura. Por ello se prepara la información antes de enseñarla, transformándola en frases más naturales. Esta es una decisión importante para el tono de la aplicación: la ventana debe parecer una explicación dirigida al usuario.

También se empieza a mostrar el criterio usado para escoger la siguiente división. En esta primera fase aparece como texto, con una línea por cada opción evaluada. Es una solución básica y permite conectar el cálculo del algoritmo con una explicación visible dentro de la interfaz. A partir de aquí, la aplicación construye el árbol y empieza a justificarlo.

5.2.3 Animación de la construcción del árbol

Con la construcción paso a paso funcionando, aparece un nuevo problema: los cambios son demasiado bruscos. Cada pulsación puede crear un nodo, mover partes del árbol y dibujar nuevas ramas. El resultado es correcto. En algunos pasos, el usuario puede perder de vista qué elemento acaba de aparecer.

Para mejorar esta lectura se añaden animaciones. Primero se animan las ramas, de manera que cada conexión parece crecer desde el punto anterior hasta el nuevo. Después se anima la aparición de los nodos, que dejan de mostrarse de golpe y pasan a aparecer de forma gradual. Estas animaciones dirigen la atención hacia el cambio producido en cada paso.

Al introducir animaciones es necesario revisar cómo se actualizan las conexiones. En las primeras versiones, al modificar el árbol se redibujan todas las ramas. Esto hace que, cada vez que aparece una conexión nueva, también se reinicien las que ya existen. Se cambia este comportamiento para conservar las ramas anteriores y animar solo las recién creadas. Gracias a ello, el avance resulta más claro y menos repetitivo.

Después se incorpora una idea pensada específicamente para explicar mejor el algoritmo: los nodos pendientes. Cuando el árbol decide dividir los datos por una característica, ya se sabe qué caminos van a aparecer y todavía falta conocer qué hay al final de cada uno. Para representar ese estado intermedio, la aplicación empieza a mostrar nodos temporales con un aspecto distinto.

Estos nodos pendientes indican que el algoritmo ya tiene una rama abierta y que aún queda por resolver su contenido. Más adelante, cuando esa parte del árbol se procesa, el nodo temporal se transforma en una decisión real o en una clasificación final. Esta transición también se anima para que el usuario perciba el cambio y entienda que se está completando una parte que antes solo queda prevista. En la Figura 5.7 se puede ver el aspecto estos nodos

pendientes.

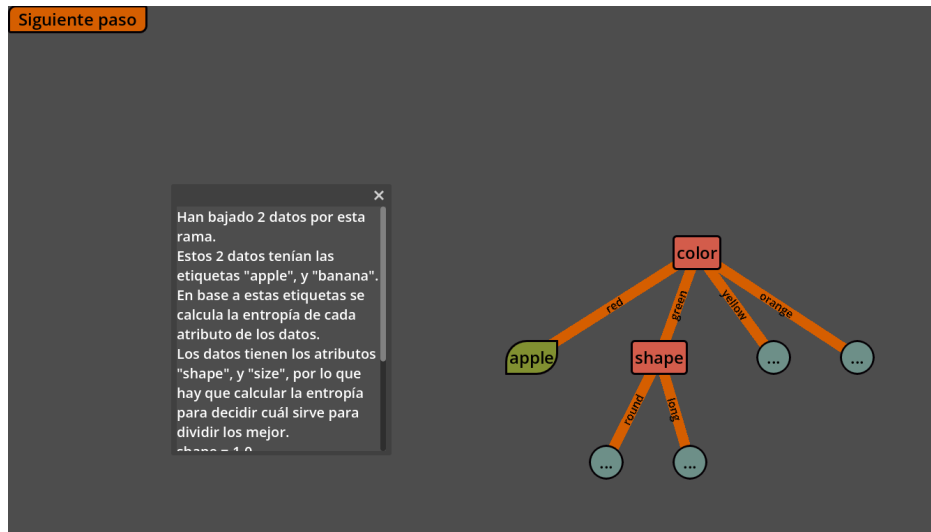


Figura 5.7: Entrenamiento del árbol en el que se aprecian los nodos pendientes.

Otra prueba temprana consiste en mostrar un valor numérico al pasar el cursor por ciertos nodos. La intención es permitir consultar información sobre la decisión tomada sin llenar todo el árbol de números. La idea funciona de forma limitada: un número aislado dice poco si no se explica qué significa. Por esa razón, más adelante esa información se traslada hacia la ventana informativa, donde puede acompañarse de una explicación más completa.

5.2.4 Gráficas y casos explicativos en la ventana

La ventana informativa evoluciona después de los primeros textos. A medida que el árbol crece y los pasos se vuelven más ricos, una única explicación en texto plano resulta insuficiente. El objetivo pasa a ser combinar texto y apoyo visual para que la decisión del algoritmo sea más fácil de seguir.

El cambio principal es introducir una gráfica de barras como la que se aprecia en la Figura 5.8. La ventana pasa de una lista de valores a una comparación visual entre las opciones que el algoritmo está evaluando. Así es más sencillo ver cuál destaca y por qué se escoge una división determinada.

Para integrar esta gráfica, la ventana pasa a tener una estructura desplazable. Esto es necesario porque algunas explicaciones ocupan más espacio que otras. El usuario puede leer la primera parte del texto, observar la gráfica y continuar después con la explicación del resultado. De esta forma, la gráfica queda dentro del relato del paso actual.

Este cambio obliga también a preparar mejor la información que recibe la ventana. Los valores utilizados para la gráfica deben conservarse como datos numéricos, sin transformarlos

desde el inicio en frases ya formateadas. Esta separación permite que la misma información sirva para explicar el paso con texto y para construir la representación visual.

En esta etapa se añade además un caso explicativo para los empates. Durante las pruebas aparece una situación en la que varias opciones pueden ser igual de buenas. Si la aplicación escoge una sin explicar nada, la decisión puede parecer arbitraria. Por eso se incorpora un texto específico que indica que hay varias alternativas equivalentes y que el algoritmo continúa con una de ellas.

También se revisan los textos de las hojas. Las primeras frases están escritas pensando en ejemplos muy pequeños, donde a veces solo llega un dato a cada resultado final. Al probar conjuntos más variados, esa redacción deja de ser válida. Se sustituye por una explicación más general, capaz de describir tanto un único ejemplo como varios ejemplos que terminan en la misma clasificación.

Este proceso se repite varias veces durante el desarrollo: al probar nuevos datos aparecen casos que la explicación inicial no cubre bien. Esos casos se van incorporando al discurso de la aplicación para que el usuario no tenga que interpretar silencios o comportamientos inesperados.

5.2.5 Primer panel de selección de datos

Hasta este momento, los datos usados para entrenar el árbol están fijados dentro del propio proyecto. Esto es cómodo para las primeras pruebas, porque permite centrarse en el funcionamiento del algoritmo y de la interfaz. Con el tiempo, esta comodidad empieza a limitar la aplicación: el usuario necesita poder escoger con qué ejemplo trabajar.

El siguiente paso es separar la elección de los datos del entrenamiento. Primero se prueba un segundo conjunto preparado manualmente, con casos diseñados para forzar ramas más profundas y situaciones menos evidentes. Esta prueba ayuda a comprobar que el árbol también funciona con ejemplos distintos del primero y que los textos siguen teniendo sentido cuando los datos cambian.

A partir de ahí se crea el panel de selección que se aprecia en la Figura 5.9. Su papel es guiar al usuario antes de empezar el entrenamiento: primero se elige una fuente de datos y después se inicia la construcción del árbol. La primera opción permite usar el conjunto de ejemplo incluido en la aplicación. La segunda opción abre la puerta a cargar un archivo CSV.

La carga de archivos se introduce de forma progresiva. En una primera versión, el panel ya muestra la opción de CSV y todavía está incompleta. Después se añade la lectura del archivo y la comprobación de que la selección sea válida antes de permitir entrenar. También se añaden mensajes de éxito o error para que el usuario sepa si puede continuar.

Con este panel, el flujo del modo algorítmico cambia de manera importante. Antes, el usuario pulsa directamente el botón de comenzar. Después de esta mejora, primero escoge

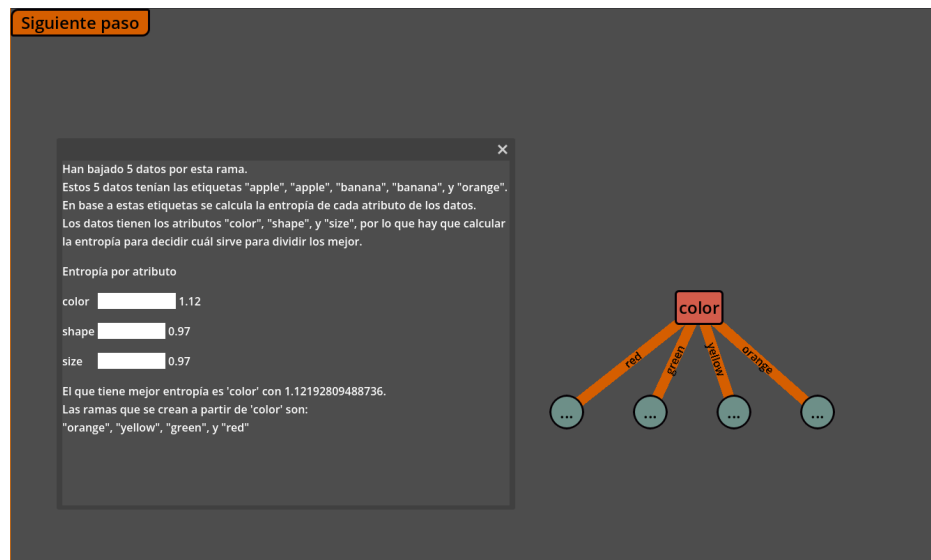


Figura 5.8: Ventana informativa en la que se dibuja la gráfica de la entropía de cada atributo.

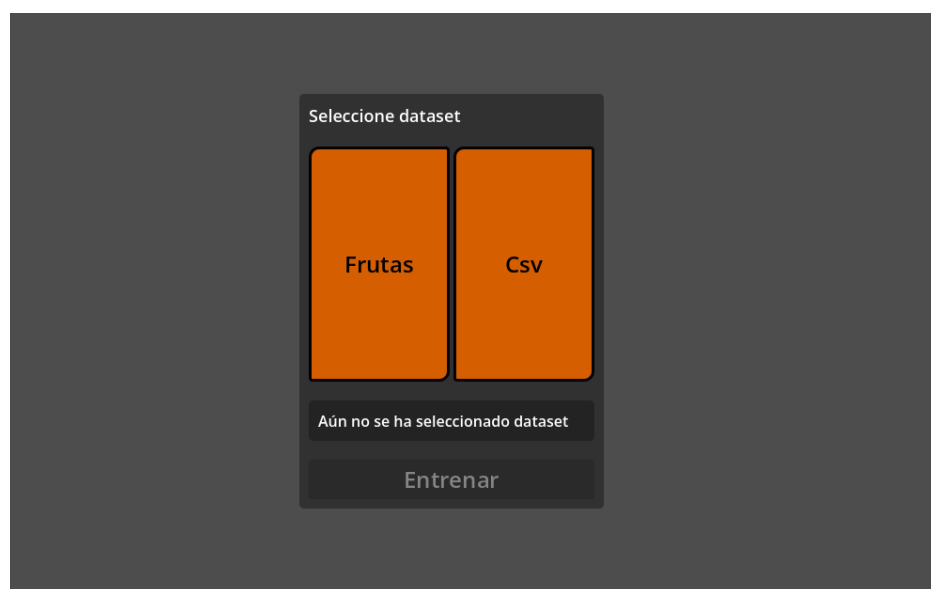


Figura 5.9: Menú que aparece tras seleccionar el modo algorítmico para seleccionar el dataset de entrenamiento.

los datos, luego confirma la selección y finalmente avanza por la construcción del árbol. Esta separación hace el proceso más comprensible, porque deja claro que el entrenamiento depende de los datos elegidos.

La versión inicial del panel todavía es provisional. Quedan detalles por ajustar, como mejorar la detección de la respuesta esperada en archivos externos y pulir la transición entre la selección de datos, el árbol y la ventana informativa. Con esas limitaciones pendientes, el cambio marca un avance importante: el árbol deja de estar ligado a un único ejemplo y empieza a comportarse como una herramienta más flexible.

5.2.6 Carga de archivos y navegación por pasos

El primer ajuste tras introducir el panel de selección es hacer que los archivos CSV puedan usarse realmente para entrenar el árbol. Hasta ese momento, la carga de datos es una posibilidad visible en la interfaz y todavía necesita más trabajo para integrarse en el flujo completo. El problema principal está en distinguir qué parte del archivo describe cada ejemplo y qué parte contiene la respuesta que el árbol debe aprender.

Para resolverlo, la aplicación empieza a buscar automáticamente la columna que representa esa respuesta. La solución se plantea de forma práctica: se reconocen algunos nombres habituales y se separan del resto de características. Con ello, el usuario puede cargar archivos sencillos sin tener que modificar manualmente la estructura de los datos antes de entrenar.

También se revisa la lectura del archivo para hacerla más fiable. El objetivo es aceptar tablas simples, con una primera fila de nombres y varias filas de ejemplos, ignorando las líneas vacías. Esta mejora sigue siendo limitada, ya que está pensada para datos formados por categorías y con una estructura clara. Con esa base basta para probar el árbol con ejemplos externos al proyecto.

El flujo visual se ajusta a esta nueva forma de empezar. Al escoger el modo algorítmico, primero aparece el panel para seleccionar los datos. El árbol y la ventana informativa permanecen ocultos hasta que el usuario hace una selección válida. Así se evita mostrar una escena vacía o una explicación sin contexto. Una vez elegidos los datos, la vista pasa directamente al entrenamiento y la ventana indica que el siguiente paso consiste en avanzar por la construcción.

En paralelo empieza a prepararse la navegación hacia atrás. Hasta ese momento, el entrenamiento solo permite avanzar. Esta limitación es poco adecuada para una herramienta educativa, porque el usuario puede querer revisar justo el paso anterior para entender por qué aparece una rama o una hoja. Por eso se añade la idea de un botón de paso previo, que se puede apreciar en la Figura ?? ¿?.

La primera versión de esta navegación hacia atrás solo prepara el camino. La aplicación empieza a recordar qué pasos ya están ejecutados y cuáles quedan pendientes, de forma que

más adelante pueda deshacer una decisión concreta. Después se añade el botón “Paso previo” a la interfaz. Con él, la construcción del árbol deja de ser estrictamente lineal: el usuario puede avanzar con “Siguiente paso” y retroceder cuando necesite revisar el proceso.

Hacer que el retroceso funcione de verdad requiere pensar qué debe ocurrir con cada tipo de elemento del árbol. Si se deshace una decisión que abre varias ramas, también hay que retirar las partes que dependen de ella. Si el elemento deshecho es temporal, resulta más claro devolverlo a su estado pendiente que eliminarlo por completo. Si se vuelve al inicio, el árbol debe quedar vacío y sin conexiones antiguas en pantalla.

Durante las pruebas aparece precisamente ese último problema. Al regresar al primer paso, algunas conexiones pueden permanecer visibles después de eliminar el árbol. La corrección consiste en actualizar también las ramas cuando el árbol queda vacío. Con esto, retroceder hasta el inicio deja la vista en un estado coherente y permite avanzar de nuevo sin arrastrar restos visuales de la ejecución anterior.

5.2.7 Primera evaluación visual del árbol

El último avance de este tramo inicial es una primera versión del modo de evaluación. Hasta entonces, la aplicación muestra cómo se construye el árbol. Todavía falta enseñar cómo se usa ese árbol para clasificar ejemplos nuevos. La evaluación se plantea de forma visual: cada ejemplo de prueba se representa como un objeto que atraviesa el árbol siguiendo las decisiones ya aprendidas.

Para introducir este modo se añaden controles específicos a la interfaz. Durante el entrenamiento, la evaluación permanece desactivada. Al terminar la construcción del árbol, el usuario puede pasar a una nueva fase y escoger datos para probar el resultado. Se reutiliza el panel de selección, ya que su papel sigue siendo el mismo: elegir con qué datos se va a trabajar.

La aplicación empieza entonces a distinguir entre dos momentos. En el primero, los datos seleccionados sirven para construir el árbol. En el segundo, los datos seleccionados sirven para comprobar cómo clasifica el árbol ya entrenado. Esta separación todavía es provisional y permite probar el ciclo completo: entrenar primero y evaluar después.

Para que la evaluación se entienda mejor, cada ejemplo se dibuja como una pieza sencilla que puede moverse por la escena. En la Figura 5.11 se ve un ejemplo de esta representación. Al pulsar el botón correspondiente, una de esas piezas entra en el árbol y se desplaza desde la raíz hasta una hoja. En cada punto sigue la rama que coincide con sus características, hasta llegar a la clasificación final.

Esta primera versión es básica. Los ejemplos se representan con formas simples y cada recorrido se ejecuta de una vez hasta el final. Las mejoras posteriores, como avanzar nivel a nivel, agrupar los ejemplos bajo las hojas o explicar cada decisión en la ventana informativa,

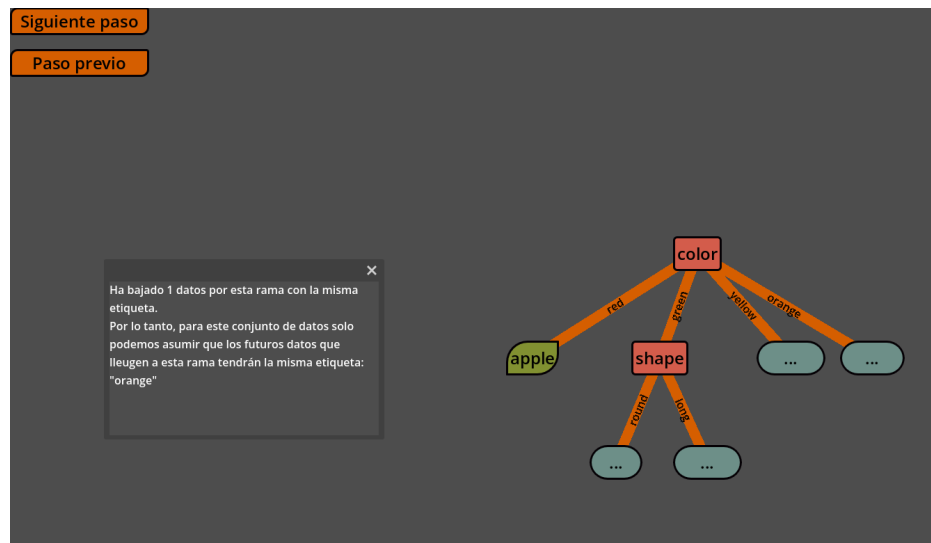


Figura 5.10: Vista del árbol en la que ahora está el botón de retroceder al paso anterior.

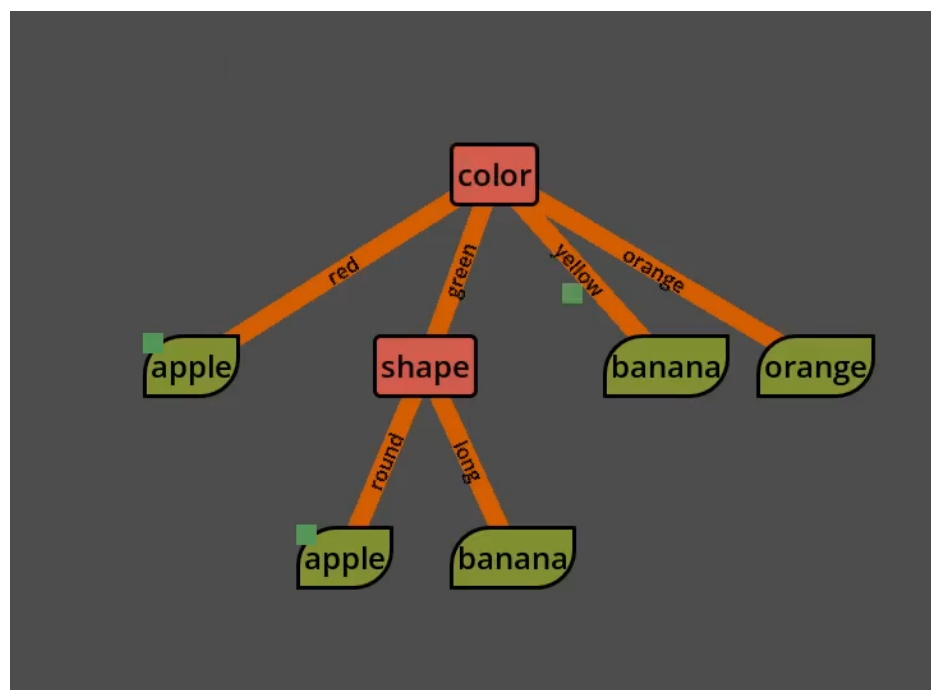


Figura 5.11: Fase de evaluación en la que se ven dos datos ya clasificados y uno en proceso de clasificación.

quedan para una fase posterior. Esta prueba deja establecida la idea central del modo de evaluación: ver físicamente cómo un ejemplo recorre las decisiones del árbol hasta llegar a una clasificación.

Con este conjunto de cambios queda completada la primera versión significativa del árbol de decisión. El proyecto pasa de dibujar elementos conectados a construir un árbol paso a paso, mostrar estados pendientes, animar la aparición de ramas y nodos, explicar las decisiones con texto y gráficos, cargar datos sencillos y ejecutar una primera evaluación visual. A partir de aquí, el desarrollo continúa con mejoras de interfaz, persistencia, adaptación a móvil y explicaciones más completas, que se tratan en apartados posteriores para separar esta primera fase de las revisiones finales.

Conclusiones

DERRADEIRO capítulo da memoria, onde se presentará a situación final do traballo, as leccións aprendidas, a relación coas competencias da titulación en xeral e a mención en particular, posibles liñas futuras,...

Capítulo 7

Trabajo futuro

Apéndices

Material adicional

EXEMPLO de capítulo con formato de apéndice, onde se pode incluír material adicional que non teña cabida no corpo principal do documento, suxeito á limitación de 80 páxinas establecida no regulamento de TFGs.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque.

Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Bibliografía

- [1] G. Sanderson, “Neural networks,” 3Blue1Brown, consultado el 11 de junio de 2026. [En línea]. Disponible en: https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi
- [2] —, “But what is a neural network?” 3Blue1Brown, consultado el 11 de junio de 2026. [En línea]. Disponible en: <https://www.3blue1brown.com/?topic=neural-networks>
- [3] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [4] D. Smilkov and S. Carter, “A neural network playground,” TensorFlow, consultado el 11 de junio de 2026. [En línea]. Disponible en: <https://playground.tensorflow.org/>
- [5] M. Kahng, N. Thorat, P. Chau, F. Viégas, and M. Wattenberg, “Gan lab: Play with generative adversarial networks in your browser!” Georgia Tech and Google Brain/PAIR, consultado el 11 de junio de 2026. [En línea]. Disponible en: <https://poloclub.github.io/ganlab/>
- [6] Interactive ML, “Interactive decision trees learning tool,” consultado el 11 de junio de 2026. [En línea]. Disponible en: <https://www.interactive-ml.com/decision-tree.html>
- [7] Godot Engine contributors, “Godot engine,” consultado el 11 de junio de 2026. [En línea]. Disponible en: <https://godotengine.org/>
- [8] —, “Godot documentation,” consultado el 11 de junio de 2026. [En línea]. Disponible en: <https://docs.godotengine.org/>
- [9] —, “Exporting for the web,” consultado el 11 de junio de 2026. [En línea]. Disponible en: https://docs.godotengine.org/en/stable/tutorials/export/exporting_for_web.html
- [10] S. Chacon and B. Straub, *Pro Git*, 2nd ed. Apress, 2014, consultado el 11 de junio de 2026. [En línea]. Disponible en: <https://git-scm.com/book/en/v2>

- [11] GitHub, Inc., “Github pages documentation,” consultado el 11 de junio de 2026. [En línea]. Disponible en: <https://docs.github.com/en/pages>